

Kit 2 - Sprint 2



Projet smartEngine

Prédiction de churn pour un SaaS B2B

Modules

Scoring prédictif et Marketing

Ciblage et Data Marketing

Préparé par Nejma HAMMOUTOU

1. Rappel et bilan du Sprint 1

1.1 Où en sommes-nous

Au Sprint 1, vous avez mis en place l'infrastructure du projet : dépôt GitHub, GEMINI.md, exploration du dataset, veille outils, brief client et première partie du dossier de conception.

Le Sprint 2 marque le passage à la production de données exploitables. Vous allez nettoyer les données brutes, construire une table analytique unique et créer les variables qui alimenteront le modèle de prédiction au Sprint 3.

1.2 Rotation des rôles

Les rôles tournent à chaque sprint. Le Scrum Master et le Product Owner du Sprint 1 cèdent leur place à deux autres membres de l'équipe. Mettez à jour votre backlog et votre GEMINI.md avec les nouveaux rôles.

1.3 Ce qui doit être fait avant de commencer

Avant de démarrer les travaux du Sprint 2, vérifiez que :

- Tous les livrables du Sprint 1 sont dans la branche main (README, veille, brief, découverte dataset, GEMINI.md, backlog, dossier de conception section 1)
- Les 5 CSV sont dans data/raw/ sur chaque machine
- Le GEMINI.md est à jour avec le sprint en cours
- Le compte-rendu de la sprint review du Sprint 1 est déposé dans docs/standups/

2. Objectifs du Sprint 2

L'objectif de ce sprint est de transformer les données brutes en une table analytique prête pour la modélisation. A la fin du sprint, vous disposez d'un fichier `data/processed/analytics.csv` avec une ligne par compte client et toutes les variables utiles pour prédire le churn.

Le sprint se décompose en trois étapes :

- **Nettoyage** : identifier et traiter les problèmes de qualité dans les données brutes (valeurs manquantes, doublons, incohérences, types incorrects)
- **Construction de la table analytique** : joindre les 5 CSV en une seule table avec une ligne par compte (`account_id`)
- **Feature engineering** : créer des variables dérivées qui capturent les signaux comportementaux du churn (tendances, ratios, agrégations temporelles)

C'est aussi le sprint où vous faites évoluer vos agents dans `.gemini/agents/` pour qu'ils prennent en charge le nettoyage et la transformation des données.

3. Mise à jour de l'infrastructure

3.1 Mise à jour du GEMINI.md

Votre GEMINI.md doit refléter l'avancement du projet. Mettez-le à jour avec :

- Le sprint en cours (Sprint 2)
- Les nouveaux rôles (nouveau SM, nouveau PO)
- Les conventions supplémentaires que vous avez définies au Sprint 1 (format des noms de fichiers, structure des rapports)
- Un résumé de ce qui a été fait au Sprint 1 (pour que l'outil ait le contexte)
- Le chemin vers la table analytique à produire : data/processed/analytics.csv

3.2 Structure du dépôt à la fin du Sprint 2

```
smartengine-groupe-X/
|-- .gitignore
|-- .gemini/
|   |-- agents/
|       |-- data-explorer.md      # agent d'exploration (Sprint 1)
|       |-- [nouvel-agent].md    # agent de traitement (Sprint 2)
|-- data/
|   |-- raw/                     # les 5 CSV (jamais modifies)
|   |-- processed/              # NOUVEAU : données transformées
|   |-- analytics.csv           # table analytique finale
|-- outputs/
|   |-- rapport-nettoyage.md     # rapport de nettoyage
|-- src/
|   |-- clean_data.py           # script de nettoyage
|   |-- build_features.py       # script de construction features
|   |-- build_analytics.py      # script de jointure table analytique
|-- docs/
|   |-- standups/
|   |-- dossier-conception.docx # section 2 ajoutée
|   |-- ...
|-- GEMINI.md                   # mis a jour Sprint 2
-- README.md
```

3.3 Evolution de vos agents

Au Sprint 1, vous avez créé un agent d'exploration (data-explorer.md) pour découvrir le dataset. Cet agent reste utile, mais les tâches du Sprint 2 (nettoyage, jointure, feature engineering) sont plus complexes et plus spécifiques.

Deux approches possibles :

- **Enrichir l'agent existant** : vous ajoutez au data-explorer.md les instructions de nettoyage, de jointure et de feature engineering. L'agent couvre alors tout le périmètre de données.
- **Créer un nouvel agent dédié** : vous gardez data-explorer.md pour l'exploration et vous créez un second agent (par exemple data-engineer.md) spécialisé dans le nettoyage et la transformation. C'est plus modulaire.

Le choix entre les deux est une décision d'équipe. Il n'y a pas de bonne réponse. Vous le justifiez dans le dossier de conception.

Livrable

Agent de traitement des données (nouveau ou enrichi)

.gemini/agents/[nom].md

Frontmatter (name, description) + corps (rôle, étapes, règles)

Le choix (enrichir vs créer) est justifié dans le dossier de conception

4. Travaux du Sprint 2

4.1 Nettoyage des données

Les données brutes ne sont jamais prêtes à être utilisées directement. Avant toute analyse ou modélisation, il faut identifier et traiter les problèmes de qualité. C'est l'étape la plus importante du projet : un modèle entraîné sur des données brutes produira des prédictions fausses.

Ce que vous devez vérifier pour chaque fichier CSV

- **Valeurs manquantes** : quels champs ont des données absentes ? Quel pourcentage ? La donnée manquante est-elle aléatoire ou systématique ?
- **Doublons** : y a-t-il des lignes identiques ? Des `account_id` qui apparaissent plusieurs fois là où ils ne devraient pas ?
- **Types incorrects** : une colonne de dates stockée en texte, un montant stocké en chaîne de caractères, un booléen stocké en entier ?
- **Valeurs aberrantes (outliers)** : des montants négatifs, des dates dans le futur, des durées d'abonnement de 999 mois ?
- **Incohérences entre fichiers** : un `account_id` présent dans `subscriptions.csv` mais absent de `accounts.csv` ? Un `churn_event` pour un compte qui a un abonnement actif ?

Comment traiter les problèmes

Pour chaque problème identifié, vous choisissez une stratégie de traitement et vous la justifiez. Il n'y a pas de bonne réponse universelle. Le choix dépend du contexte métier et du volume concerné.

Problème	Stratégies possibles	Quand l'utiliser
Valeurs manquantes	Suppression de la ligne, imputation par la médiane, imputation par la moyenne, imputation par le mode, valeur par défaut	Suppression si < 5% des lignes. Imputation si la colonne est importante pour le modèle.
Doublons	Suppression, conservation du plus récent, agrégation	Suppression sauf si le doublon porte une information (ex : deux tickets pour le même compte = normal).
Types incorrects	Conversion de type (<code>to_datetime</code> , <code>astype</code>), nettoyage de format	Systématique. Les types doivent être corrects avant toute opération.
Outliers	Suppression, winsorisation (plafonnement), conservation avec flag	Suppression si erreur de saisie. Conservation si valeur extrême mais réelle.
Incohérences inter-fichiers	Suppression des orphelins, correction manuelle, flag	Toujours documenter. Ne jamais ignorer silencieusement.

Tâche de recherche

Avant de nettoyer les données, documentez dans votre base de connaissances :

Qu'est-ce que l'imputation de données ? Quelles sont les méthodes les plus courantes (moyenne, médiane, mode, KNN, régression) ? Dans quels cas utiliser l'une plutôt que l'autre ?

Qu'est-ce qu'un outlier ? Comment le détecter (IQR, z-score, visualisation) ? Faut-il toujours le supprimer ?

Pourquoi est-il important de documenter chaque décision de nettoyage ? Quel lien avec la reproductibilité d'un projet data ?

Le rapport de nettoyage

Chaque décision de nettoyage doit être tracée dans un rapport. Ce rapport est un livrable du sprint. Il doit permettre à quelqu'un qui n'a pas participé au projet de comprendre exactement ce qui a été fait et pourquoi.

Pour chaque fichier CSV, le rapport indique :

- Les problèmes identifiés (avec chiffres : X lignes concernées, Y% de valeurs manquantes)
- La stratégie choisie pour chaque problème
- La justification de ce choix
- Le résultat après traitement (nombre de lignes avant/après, pourcentage de données conservées)

Livrable

Rapport de nettoyage

outputs/rapport-nettoyage.md

Un tableau par fichier CSV : problème, volume, stratégie, justification, résultat

Section finale : bilan global (nombre de lignes conservées, taux de perte)

4.2 Construction de la table analytique

Les 5 fichiers CSV contiennent des informations à des granularités différentes : `accounts.csv` est au niveau du compte, `feature_usage.csv` est au niveau du mois, `support_tickets.csv` est au niveau du ticket. Pour entraîner un modèle de prédiction de churn, il faut ramener toutes ces informations à une granularité unique : une ligne par compte.

La table analytique (`data/processed/analytics.csv`) est le résultat de cette agrégation. Chaque ligne représente un compte client, et chaque colonne est une variable qui décrit ce compte.

Les étapes de construction

Partez d'une table de référence. Puis enrichissez-la en joignant les informations des 4 autres fichiers :

- **Depuis `subscriptions.csv`** : durée de l'abonnement, type de plan actuel, nombre de changements de plan (upgrades/downgrades), ancienneté
- **Depuis `feature_usage.csv`** : agrégez par compte les métriques d'usage (moyenne, tendance, dernière valeur). C'est la partie la plus riche en features potentielles.
- **Depuis `support_tickets.csv`** : nombre total de tickets, nombre de tickets critiques, délai moyen de résolution, dernier ticket
- **Depuis `churn_events.csv`** : la variable cible. Le compte a-t-il churné (1) ou non (0). Attention à la temporalité : ne comptez que les churns qui se sont produits après la période d'observation des features.

Variable cible

La variable cible (churn oui/non) est la colonne que le modèle cherchera à prédire au Sprint 3. Assurez-vous qu'elle est binaire et correctement alignée avec les features. Un compte churné en janvier ne doit pas avoir de features de février.

Livrable

Table analytique

`data/processed/analytics.csv`

4.3 Feature engineering

Le feature engineering est l'étape où vous créez de nouvelles variables à partir des données existantes. C'est souvent là que se joue la qualité d'un modèle.

L'idée est de capturer des signaux comportementaux que le modèle pourra exploiter. Par exemple, un client dont l'usage baisse mois après mois est probablement plus à risque de churn qu'un client dont l'usage est stable.

Types de features à considérer

Catégorie	Exemples	Source
Comportement d'usage	Nombre moyen de sessions, features utilisées, tendance d'usage sur 3 mois	feature_usage.csv
Engagement	Jours depuis la dernière connexion, ratio jours actifs / jours totaux	feature_usage.csv
Support	Nombre de tickets, ratio tickets critiques / total, délai moyen résolution	support_tickets.csv
Abonnement	Ancienneté en mois, nombre de changements de plan, type de plan	subscriptions.csv
Financier	MRR du compte, évolution du MRR, écart par rapport à la moyenne du plan	accounts.csv + subscriptions.csv
Temporel	Mois d'inscription, saison, ancienneté au moment de l'observation	accounts.csv + subscriptions.csv

Ce tableau n'est pas exhaustif. C'est à vous de décider quelles features créer en fonction de votre compréhension du dataset et du problème métier. Chaque feature doit être justifiée dans le dossier de conception : pourquoi cette variable pourrait-elle aider à prédire le churn ?

Tâche de recherche

Le feature engineering est au coeur de la data science appliquée au marketing. Documentez :

Qu'est-ce que le feature engineering ? Pourquoi est-ce souvent plus important que le choix de l'algorithme ?

Qu'est-ce qu'une variable de tendance (trend feature) ? Comment la calculer à partir de données temporelles ?

Qu'est-ce que l'encodage des variables catégorielles (one-hot encoding, label encoding) ? Quand utiliser l'un ou l'autre ?

Qu'est-ce que la normalisation et la standardisation ? Pourquoi certains algorithmes en ont besoin et d'autres non ?

Livable

Script de feature engineering
src/build_features.py

5. Dossier de conception : section 2

Ajoutez une deuxième section à votre dossier de conception existant (docs/dossier-conception.docx). Cette section couvre le traitement des données.

5.1 Contenu attendu

La section 2 du dossier de conception doit répondre aux questions suivantes :

- **Etat des données brutes** : quel était l'état de chaque fichier CSV avant nettoyage ? (résumé du rapport de nettoyage)
- **Décisions de nettoyage** : quelles stratégies avez-vous choisies et pourquoi ? Quelles alternatives avez-vous écartées ?
- **Construction de la table analytique** : comment avez-vous joint les 5 fichiers ? Quelle granularité ? Combien de lignes et de colonnes dans le résultat ?
- **Features créées** : liste des variables ajoutées, leur source, leur justification métier
- **Agent de traitement** : comment l'agent a-t-il été configuré ? Quelles itérations ont été nécessaires ? Qu'est-ce qui a bien fonctionné et qu'est-ce qui a dû être corrigé manuellement ?

Tâche de recherche

La qualité des données est un enjeu central du RGPD et de la gouvernance data. Documentez :

Qu'est-ce que la qualité des données ? Quelles dimensions la définissent (complétude, exactitude, cohérence, fraîcheur, unicité) ?

Pourquoi faut-il documenter le processus de nettoyage ? Quel lien avec l'auditabilité et la conformité RGPD ?

Qu'est-ce qu'un pipeline de données ? Pourquoi est-il important que le nettoyage soit reproductible (script vs modifications manuelles) ?

Livrable

Section 2 du dossier de conception

docs/dossier-conception.docx (section ajoutée)

Décisions de nettoyage justifiées

Features documentées avec justification métier

Retour d'expérience sur l'utilisation de l'agent

6. Backlog Sprint 2

Mettez à jour votre backlog avec les tâches du Sprint 2. Les user stories du Sprint 1 doivent passer au statut 'Done'. Les nouvelles stories du Sprint 2 sont ajoutées.

Exemples de user stories Sprint 2 :

- En tant que développeur IA, je veux identifier les valeurs manquantes dans chaque CSV afin de choisir une stratégie de traitement adaptée.
- En tant que Product Owner, je veux une table analytique avec une ligne par compte afin que le modèle de prédiction puisse être entraîné au Sprint 3.
- En tant que Scrum Master, je veux un rapport de nettoyage documenté afin de garantir la traçabilité des décisions.

Livrable

Backlog Sprint 2

Même outil qu'au Sprint 1

Stories Sprint 1 passées en Done

Nouvelles stories Sprint 2 ajoutées et assignées

7. Livrables

Livrable	Fichier attendu	Responsable
Agent de traitement (nouveau ou enrichi)	.gemini/agents/[nom].md	Développeurs IA
Rapport de nettoyage	outputs/rapport-nettoyage.md	Toute l'équipe
Scripts de traitement	src/clean_data.py, src/build_features.py, src/build_analytics.py	Générés par l'agent
Table analytique	data/processed/analytics.csv	Toute l'équipe
Section 2 dossier de conception	docs/dossier-conception.docx	Toute l'équipe
GEMINI.md mis à jour	GEMINI.md (racine)	Scrum Master
Backlog Sprint 2	Lien outil de gestion	Scrum Master

8. Rituels Scrum

Les rituels restent identiques au Sprint 1.

8.1 Daily Standup

10 minutes maximum en début de séance. Trois questions : qu'ai-je fait, que vais-je faire, qu'est-ce qui me bloque. Compte-rendu dans docs/standups/AAAA-MM-JJ.md.

8.2 Sprint Review

A la fin du sprint, vous présentez :

- La table analytique : combien de lignes, combien de features, quelles décisions de nettoyage
- Le rapport de nettoyage : les problèmes trouvés et les stratégies choisies
- L'agent : comment il a été configuré, ce qui a marché, ce qui a dû être ajusté
- Le dossier de conception section 2 : les justifications des choix

9. Ressources

Ressource	Lien
Guide Orchestration d'agents IA	Discord #liens-outils
Documentation Gemini CLI agents	geminicli.com/docs/core/subagents/
pandas : gestion des valeurs manquantes	pandas.pydata.org/docs/user_guide/missing_data
pandas : merge et join	pandas.pydata.org/docs/user_guide/merging
scikit-learn : preprocessing	scikit-learn.org/stable/modules/preprocessing